
Proyecto 1 – Introducción a la Programación y Computación 2

202308221 – Jose Juan Laynez Zapeta

Resumen

El siguiente documento presenta información relevante acerca del proyecto 1 del curso de introducción a la programación 2.

Con este proyecto, se pretende aprender diversos temas de programación como programación orientada a objetos, arquitectura de software, manipulación de archivos XML, creación de graficos en Graphviz, y como punto fuerte, el aprendizaje de datos abstractos.

Igualmente, el contenido del texto es de naturaleza técnica, así que se verá de forma objetiva y profunda más acerca de este.

Palabras clave

Programación, Tipos de Datos Abstractos, Clases, Objetos, manipulación de archivos.

Abstract

The following document presents relevant information regarding the first project of the Introduction to Programming 2 course.

This project aims to cover various programming topics such as object-oriented programming, software architecture, XML file manipulation, and graph creation using Graphviz—with a particular focus on learning about abstract data types.

Likewise, the content of the text is technical in nature, so it will be examined in an objective and in-depth manner.

Keywords

Programming, Abstract Data types, classes, objects, file manipulation.

Introducción

A continuación, se presentará mas información acerca del proyecto. Este es un sistema de para una empresa llamada Chapin Warriors S.A, y con la cual se pretende brindar una solución para sus drones ChapinEyes, ChapinFighter y ChapinRescue.

Para ello, se tuvo que trabajar con diversos conceptos de programación, tales como TDA's (Tipos de Datos Abstractos), Programación Orientada a Objetos, manipulación de archivos XML, generación de gráficos con Graphviz, y la implementación de algoritmos de búsqueda de caminos en mallas.

El proyecto consiste en desarrollar un sistema de control que permita gestionar misiones de rescate de unidades civiles y extracción de recursos en ciudades que se encuentran en zonas de conflicto. Para lograr esto, fue necesario implementar estructuras de datos personalizadas sin usar las libreríasde C#, leer y procesar archivos de configuración en formato XML, y generar reportes gráficos que muestren las rutas que deben seguir los robots para completar sus misiones.

Se desarrolló en Visual Studio Code y Visual Studio.

Desarrollo del tema

El proyecto fue elaborado con base en los siguientes aspectos fundamentales:

a. Implementación de Tipos de Datos Abstractos (TDA):

Una de las partes más importantes del proyecto fue la creación de nuestras propias estructuras de datos. Como no podíamos usar las listas, colas o pilas que ya trae C#, tuvimos que crearlas desde cero usando clases y nodos.

Para esto, implementé principalmente listas enlazadas simples para almacenar:

- La lista de ciudades cargadas desde el archivo XML
- La lista de robots disponibles (ChapinRescue y ChapinFighter)
- La lista de celdas que conforman cada ciudad
- La lista de mallas de tablero que representan el mapa de cada ciudad

Cada una de estas estructuras tiene sus propios nodos. Por ejemplo, para la lista de ciudades usé un nodo llamado CiudadNodo que contiene una referencia a un objeto Ciudad y un puntero al siguiente nodo. Esto me permitió tener un control total sobre cómo se almacenan y manipulan los datos en memoria.

La malla de la ciudad fue quizás la estructura más compleja, ya que requiere una implementación de matriz dispersa con listas de cabeceras tanto para filas como para columnas. Esto permite acceder de manera eficiente a cualquier celda del mapa usando sus coordenadas (fila, columna).

b. Programación Orientada a Objetos:

El diseño del sistema se basó completamente en POO. Creé varias clases principales:

- Celda: Representa cada posición en el mapa de la ciudad. Tiene propiedades como Fila, Columna, Tipo (que puede ser Intransitable, Camino, PuntoEntrada, UnidadCivil, Recurso), CapacidadMilitar (para las celdas con unidades militares), y un booleano Visitada que usamos para los algoritmos de búsqueda.

- Robot (clase abstracta): De esta heredan dos clases concretas:
- ChapinRescue: Robots diseñados para rescatar civiles. No pueden enfrentar unidades militares.
- ChapinFighter: Robots de combate que pueden extraer recursos y tienen una capacidad de combate que disminuye cada vez que derrotan una unidad militar.
- Ciudad: Contiene el nombre, número de filas y columnas, y la lista de celdas que conforman el mapa.
- MallaTablero: Implementa la estructura de la matriz dispersa con sus cabeceras de fila y columna.
- Controller: Esta clase actúa como intermediario entre la interfaz gráfica y el modelo. Se encarga de coordinar todas las operaciones: cargar el archivo XML, buscar ciudades, obtener puntos de entrada, civiles y recursos, y ejecutar las misiones.
- Model: Contiene el estado global del sistema con todas las listas principales (ciudades, robots, mallas).

c. Manipulación de archivos XML

Para cargar la configuración del sistema, fue necesario implementar un lector de archivos XML. El archivo tiene una estructura específica que incluye:

1. Una lista de ciudades, donde cada ciudad tiene:
 - Un nombre con atributos de filas y columnas
 - Múltiples etiquetas "fila" que contienen el mapa como una cadena de caracteres (* para intransitable, espacio para camino, E para punto de entrada, C para unidad civil, R para recurso)
 - Etiquetas "unidadMilitar" que especifican la posición y capacidad de combate de las unidades militares
2. Una lista de robots con su nombre, tipo (ChapinRescue o ChapinFighter) y capacidad (solo para ChapinFighter)

El proceso de lectura implica parsear el XML, crear los objetos correspondientes y almacenarlos en nuestras estructuras de datos personalizadas. Un detalle importante es que si se cargan varios archivos y hay nombres repetidos, los datos deben actualizarse en lugar de duplicarse.

d. Algoritmos de búsqueda de caminos

Esta fue probablemente la parte más desafiante del proyecto. Para encontrar rutas desde un punto de entrada hasta el objetivo (ya sea una unidad civil o un recurso), implementé algoritmos de búsqueda con retroceso.

Para las misiones de rescate (ChapinRescue):

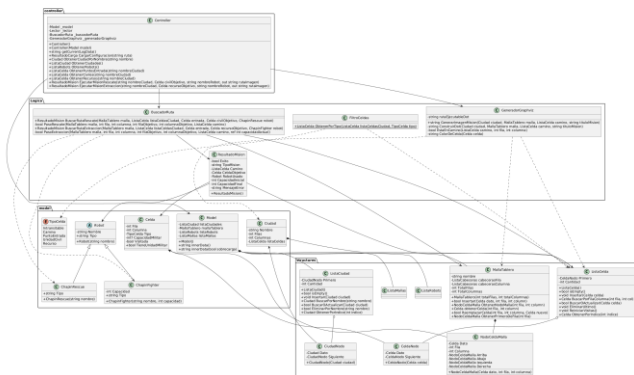


Diagrama de clases del sistema de control Chapin Warriors. Fuente:
Elaboración propia mediante PlantUML.

- El robot debe evitar completamente las unidades militares
- Solo puede transitar por celdas de tipo Camino, PuntoEntrada y UnidadCivil
- Las celdas de tipo Recurso son intransitables para este tipo de misión
- El algoritmo explora recursivamente en las cuatro direcciones (arriba, abajo, izquierda, derecha) marcando las celdas como visitadas para evitar ciclos
- Si no encuentra camino, hace retroceso (backtracking) desmarcando las celdas y probando otras rutas

Para las misiones de extracción (ChapinFighter):

- El robot puede enfrentar unidades militares si su capacidad de combate es mayor
- Cada vez que derrota una unidad militar, su capacidad disminuye en la cantidad de la capacidad del enemigo derrotado
- Debe verificar en cada paso si tiene suficiente capacidad para continuar
- El algoritmo es similar al de rescate, pero con la lógica adicional de gestionar la capacidad de combate

Ambos algoritmos retornan una lista de celdas que representan el camino encontrado, o indican "Misión Imposible" si no existe ruta válida.

Una vez encontrada la ruta de la misión, el sistema debe generar una imagen que muestre visualmente el mapa de la ciudad con el camino marcado. Para esto, usé Graphviz.

El proceso consiste en:

1. Crear un archivo DOT que describe el camino recorrido por el robot, que puede variar por tipo de misión.
2. En este caso, creamos una tabla HTML dentro del DOT donde cada celda es una celda de la tabla con un color específico:
 - Negro: celdas intransitables
 - Verde: puntos de entrada
 - Blanco: caminos normales
 - Rojo: caminos con unidades militares
 - Azul: unidades civiles
 - Gris: recursos
 - Beige (el cual se llama Burlywood 2 internamente): para marcar el camino que sigue el robot
3. Ejecutar el comando de Graphviz (dot.exe) que convierte el archivo DOT en una imagen PNG
4. Mostrar la imagen al usuario junto con los detalles de la misión
- 5.
- e. Dificultades encontradas:

de los mayores retos a la hora de realizar el programa fue la elaboración de la creación del algoritmo de retrocesos para la elaboración del camino, ya que se tenían que trabajar con dos lógicas (similares, en lo que se refiere al objetivo y movimientos, pero diferentes en cómo actúan contra.

Finalmente, una vez que cada componente estuvo desarrollado de manera aislada, el siguiente gran reto fue la integración de todos los módulos. Conectar el lector de XML con los TDA, y a su

vez con el algoritmo de búsqueda y el generador de Graphviz, requirió un cuidado especial para asegurar que los datos fluyeran correctamente sin perder referencias en memoria dinámica.

Conclusiones

Después de completar este proyecto, puedo destacar varias conclusiones importantes:

1. **Aprendizaje profundo de estructuras de datos:** Al tener que implementar nuestras propias listas y estructuras sin usar las librerías de C#, realmente entendí cómo funcionan por dentro estos tipos de datos. Ya no son "cajas negras" sino que sé exactamente qué pasa en memoria cuando inserto, busco o elimino un elemento.
2. **Importancia de la abstracción:** El uso de programación orientada a objetos me permitió modelar el problema de manera más clara. Cada clase tiene una responsabilidad bien definida, lo que hace el código más mantenible y fácil de entender.
3. **Desafíos de los algoritmos de búsqueda:** Implementar el backtracking fue complicado al principio, especialmente manejar correctamente el marcado y desmarcado de celdas visitadas. Sin embargo, una vez que lo entendí, vi lo poderoso que es este enfoque para problemas de búsqueda de caminos.
4. **Manejo de archivos y persistencia:** Aprender a leer archivos XML y transformar esa información en objetos fue una habilidad muy útil. Entendí la importancia de validar los datos de entrada y manejar posibles errores.
5. **Integración de herramientas externas:** Usar Graphviz me mostró cómo podemos aprovechar herramientas externas para

mejorar la presentación de nuestros programas. Aunque al principio fue complicado configurar la ruta del ejecutable y entender el formato DOT, el resultado visual vale totalmente la pena.

6. **Gestión de versiones con Git:** El requisito de hacer 4 releases en GitHub me obligó a llevar un control adecuado de versiones. Esto me ayudó a organizar mejor el desarrollo y a no perder cambios importantes.

Referencias bibliográficas

Máximo 5 referencias en orden alfabético.

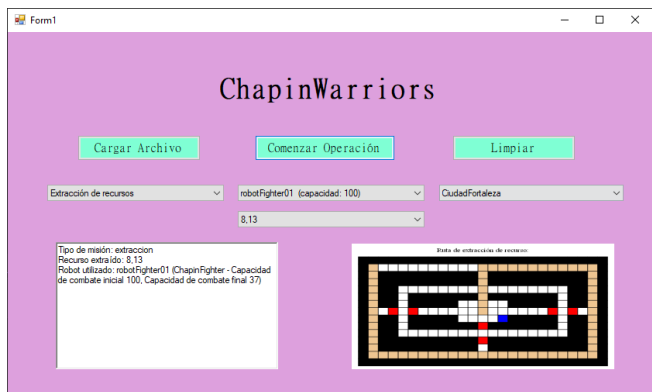
Joyanes Aguilar, L. (2008). Fundamentos de Programación: Algoritmos, Estructura de Datos y Objetos. McGraw-Hill Education.

Documentación oficial de C# y .NET. Microsoft. Disponible en: <https://docs.microsoft.com/es-es/dotnet/csharp/>

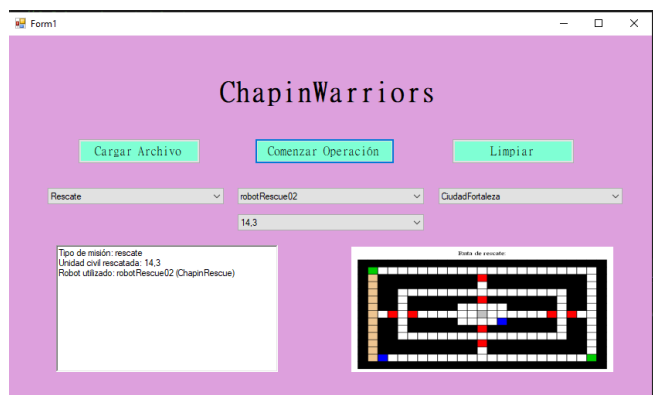
Documentación de Graphviz. AT&T Research Labs. Disponible en: <https://graphviz.org/documentation/>

Material de clase del curso Introducción a la Programación y Computación 2, USAC, 2do semestre 2026.

Extensión:

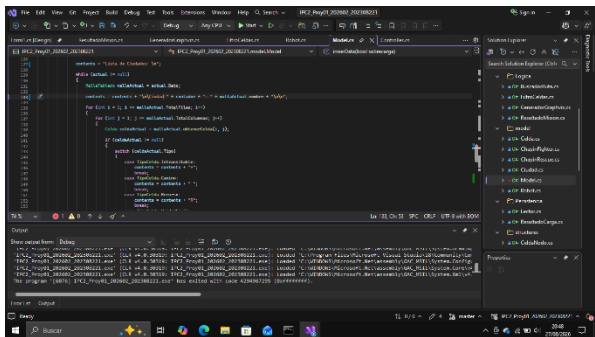


Interfaz del sistema de control tras la carga exitosa del archivo de configuración XML. *Fuente: Elaboración propia.*



Salida gráfica de una misión de rescate exitosa, mostrando el camino seguro trazado hacia la unidad civil.

Fuente: Elaboración propia (Generada con Graphviz).



Parte de código
Fuente: Elaboración propia